

Terraform

Declarative infrastructure-as-code, state management, and multi-cloud provisioning

Overview

Terraform (HashiCorp) is a declarative IaC tool using HCL to define, plan, and apply infrastructure across providers (AWS, Azure, GCP, Kubernetes, and hundreds more) with a consistent workflow. State tracking makes it distinct from purely imperative scripting.

Core Workflow

	Purpose
<code>terraform init</code>	Downloads providers/modules, configures backend
<code>terraform plan</code>	Computes diff between desired config and current state — no changes applied
<code>terraform apply</code>	Executes the plan, creates/updates/destroys real resources
<code>terraform destroy</code>	Tears down all resources tracked in state
<code>terraform validate</code>	Checks configuration syntax and internal consistency
<code>terraform fmt</code>	Auto-formats HCL to canonical style

State Management

- **Remote backend** — store state in Azure Storage / S3 / Terraform Cloud, never commit to git
- **State locking** — prevents concurrent applies corrupting state (native in Azure Storage + S3/DynamoDB combo)
- **Workspaces** — isolate state per environment (dev/staging/prod) within one configuration
- **terraform import** — brings existing, manually-created resources under Terraform management
- **Drift detection** — `plan` against real infra surfaces out-of-band manual changes

Language Building Blocks

	Purpose
<code>provider</code>	Configures the target platform (auth, region/subscription)
<code>resource</code>	Declares a manageable infrastructure object
<code>data</code>	Reads existing infrastructure without managing its lifecycle
<code>variable</code> / <code>output</code>	Parameterize input and expose values for chaining modules
<code>module</code>	Reusable, composable bundle of resources
<code>locals</code>	Named expressions for DRY configuration

Security Best Practices

- **Never hardcode secrets** — pull from Key Vault/Secrets Manager via data sources at plan time
- **Least-privilege CI identity** — the service principal/role running apply should be scoped tightly, with plan-only access on PRs
- **State encryption** — enable encryption at rest on the remote backend; state can contain sensitive attributes
- **Policy as code** — Sentinel (Terraform Cloud/Enterprise) or Open Policy Agent (OPA) / Checkov for pre-apply guardrails
- **Module pinning** — pin provider and module versions to avoid unreviewed drift on re-init
- **Plan review gate** — require human/automated review of the plan output before apply in CI/CD

CI/CD Pattern

PR opened → pipeline runs `fmt`, `validate`, `plan`, policy scan (Checkov/tfsec) → plan output posted as PR comment for review → on merge, pipeline runs `apply` using a scoped service principal with remote state locking to prevent concurrent runs.

Common Pitfalls

Fix
Always use a remote backend with locking and encryption at rest

Local state file committed to git

Manual console changes causing drift

Enforce IaC-only changes via policy; run scheduled drift-detection plans

No plan review before apply in CI/CD

Require plan output review/approval gate before merge-triggered apply

Unpinned provider/module versions

Pin versions with pessimistic constraints; review changelogs before bumping

Monolithic single-state configuration

Split by blast radius/lifecycle (network, platform, app) into separate state files

Interview / Exam Quick Hits

- `terraform plan` is **read-only** against the provider API; only `apply` mutates infrastructure
- `count` and `for_each` both create multiple resource instances — `for_each` is preferred for stability when items can be added/removed from the middle of a set
- Provider version constraints use **pessimistic operator** (`~>`) to allow patch updates without breaking changes
- `terraform state mv` refactors resource addresses without destroying/recreating infrastructure
- Remote state **output values** can be consumed cross-configuration via `terraform_remote_state` data source